>_ MCMP113 · Week 12

Functions (Methods): Definition, Calling, Parameter Passing

Why methods, anatomy of a method, the call stack, passing by value, arrays as parameters, ref and out, default and named arguments, overloading

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 12 slides and add what does not fit on a slide: the reasons behind the rules, call-stack diagrams for every way of passing a parameter, the thirteen example programs (complete, or as an excerpt for the longer ones) with real sample runs, the common mistakes, a glossary and graded exercises. Every listing is read from the tested program in the `code` folder (`12_01` ... `12_13`); open it in VS Code and run it while you read. Typed input in the sample runs is shown in **amber**. From this week on you may (and should) split every program into methods. Call traces with value and `ref` parameters are typical final-exam questions.

Contents

1 Learning outcomes **2**

2 Why methods? **2**

2.1 Example 1 – a receipt without copy-paste . . . 3

3 Defining a method **4**

3.1 Anatomy . . . 4

3.2 Where methods go in a top-level program . . . 4

3.3 Example 2 – void and value-returning methods . . . 5

4 Calling a method: the call stack **6**

4.1 Six steps of a call . . . 6

4.2 Example 3 – a program that prints its own call stack . . . 7

4.3 Parameters and arguments . . . 8

5 Passing parameters **8**

5.1 By value: the method gets a copy . . . 8

5.2 Arrays as parameters: the reference is copied . . . 9

5.3 ref and out . . . 10

5.4 Summary: four ways to pass . . . 11

6 Flexible methods **12**
6.1 Example 7 – optional parameters and named arguments 12
6.2 Example 8 – overloading 13

7 Refactoring earlier programs **14**
7.1 Example 9 – a math toolkit 14
7.2 Example 10 – input methods 15
7.3 Example 11 – an array toolkit 16
7.4 Example 12 – a menu calculator 17

8 Debugging calls in VS Code **18**

9 Common mistakes **19**

10 Summary **20**

11 Glossary (English – Turkish) **20**

12 Exercises **21**
12.1 Check yourself: predict the output 21
12.2 In-class exercises (pairs) 21
12.3 Homework 21
12.4 Extra practice 22

13 References **22**

>_ 1 Learning outcomes

After this week you should be able to:

1. explain why programs are split into methods (DRY, decomposition, testing, reuse);
2. define a method with return type, name, parameter list and body, and place it as a local function at the bottom of `Program.cs`;
3. distinguish `void` methods from value-returning methods and call each correctly;
4. trace a call on the **call stack**: arguments are copied into parameters, a new frame is created and removed on return;
5. predict the effect of passing by value, passing an array, and passing with `ref` and `out`;
6. use optional parameters, named arguments and method overloading;
7. refactor earlier programs into methods and step into calls with the VS Code debugger.

>_ 2 Why methods?

Since Week 3 you have *called* methods written by others: `Console.WriteLine`, `int.Parse`, `Math.Sqrt`, `int.TryParse`. This week you write your own. A **method** (in other languages:

function, procedure, subroutine) is a named block of code that does one job. It runs only when it is **called**; data goes in through **parameters** and a result can come back with `return`. There are four reasons to use methods:

- **DRY – Don't Repeat Yourself.** Code that is copied five times must be fixed five times. A method is written once and called five times.
- **Decomposition.** A big problem becomes several small, named steps. A *structure chart* (a tree of methods) is a design you can draw before coding; in a flowchart (Week 2) the *predefined-process* box is exactly a method call.
- **Testing.** A small method with clear inputs and one output can be checked with known values.
- **Reuse and readability.** `if (IsLeapYear(year))` explains itself; the long Boolean expression does not.

2.1 Example 1 – a receipt without copy-paste

In Week 6 every line of = signs needed its own `for` loop. Now the loop lives in `PrintLine`; each line of the receipt is one call with two **arguments**: which symbol and how many.

Listing 1: 12_01_WhyMethods/Program.cs

```
// MCMP113 · Week 12 · Example 1 — Why methods? Write the code once, call it many times
// The loop that draws a line is written ONCE (PrintLine) and called with different arguments.

PrintLine('=', 30);
Console.WriteLine("          OKAN CAFE");
PrintLine('=', 30);
PrintItem("Tea x2", 60);
PrintItem("Simit x1", 25);
PrintItem("Toast x1", 110);
PrintLine('-', 30);
PrintItem("TOTAL", 60 + 25 + 110);
PrintLine('=', 30);
PrintLine('*', 12);

// prints symbol count times, then a new line
static void PrintLine(char symbol, int count)
{
    for (int i = 0; i < count; i++)
    {
        Console.Write(symbol);
    }
    Console.WriteLine();
}

// prints one receipt line (30 characters wide)
static void PrintItem(string name, double amount)
{
    Console.WriteLine($"{name,-18}{amount,9:F2} TL");
}
```

Output

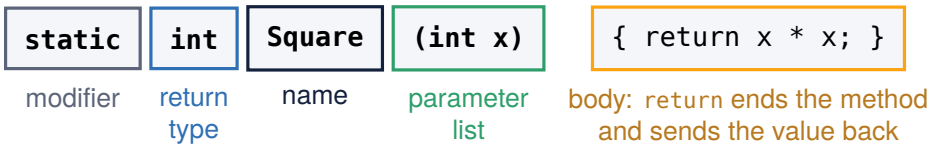
```
==============================
          OKAN CAFE
==============================
Tea x2                  60.00 TL
Simit x1                25.00 TL
Toast x1               110.00 TL
------------------------------
TOTAL                  195.00 TL
==============================
***********
```

call	parameters get
`PrintLine('=', 30)`	`symbol='=', count=30`
`PrintLine('*', 12)`	`symbol='*', count=12`
`PrintItem("TOTAL", 60 + 25 + 110)`	`name="TOTAL", amount=195`

An argument may be an expression: `60 + 25 + 110` is evaluated *first*, and only the value 195 is passed.

>_ 3 Defining a method

3.1 Anatomy



- **`static`**: the local function uses only its parameters and cannot see the variables of the main program (error CS8421 "A static local function cannot contain a reference to ..."). We always write it: all data comes in through parameters and goes out with `return`.
- **Return type**: the type of the result (`int`, `double`, `bool`, `string`, `int[]` ...), or `void` if there is none.
- **Name**: PascalCase; a verb for actions (`PrintLine`, `ReadInt`, `Swap`), a noun for computed values (`Square`, `Average`), `Is`/`Has` for yes/no questions (`IsPrime`).
- **Parameter list**: type and name of every input, separated by commas; `()` if there is none.
- **Body**: the statements between the braces. In a value-returning method every path must end with `return` (Week 13).

3.2 Where methods go in a top-level program

Our programs use **top-level statements** (Week 3). A method declared inside them is a **local function**. **Course rule:** local functions go at the *bottom* of `Program.cs`, after all statements, each with a one-line `//` comment. They do not run by themselves: execution starts at the first statement, and a method body runs only when it is called; the order of the methods does not matter. The compiler turns the file into the classic form you will see in books:

Program.cs (our style)

```
int y = Square(5);
Console.WriteLine(y);

// returns x * x
static int Square(int x)
{
    return x * x;
}
```

what the compiler builds (classic)

```
class Program
{
    static void Main()
    {
        int y = Square(5);
        Console.WriteLine(y);
    }
    static int Square(int x) { ... }
}
```

3.3 Example 2 – void and value-returning methods

`Greet` is a `void` method: it *does* something (prints) and gives nothing back, so its call is a statement on its own line. `Square`, `Cube` and `CircleArea` *compute* something and give it back with `return`; their calls are expressions with a value that can be printed, stored or combined: `Square(3) + Square(4)` is $9 + 16 = 25$. `Cube` calls `Square`: methods can call methods. The argument `r` and the parameter `radius` have different names; that is normal.

Listing 2: 12_02_FirstMethods/Program.cs

```
// MCMP113 · Week 12 · Example 2 — Anatomy of a method: define once, call many times
// Value-returning methods (Square, Cube, CircleArea) and a void method (Greet).

Greet("Elif");
Greet("Mehmet");

Console.Write("Enter an integer: ");
int n = int.Parse(Console.ReadLine());
Console.WriteLine($"Square({n}) = {Square(n)}");
Console.WriteLine($"Cube({n}) = {Cube(n)}");

int sum = Square(3) + Square(4);  // calls in an expression
Console.WriteLine($"Square(3) + Square(4) = {sum}");

Console.Write("Radius in cm: ");
double r = double.Parse(Console.ReadLine());
double area = CircleArea(r);  // store the result
Console.WriteLine($"Area = {area:F2} cm^2");

// prints a greeting; void = it returns nothing
static void Greet(string name)
{
    Console.WriteLine($"Hello, {name}! Welcome to MCMP113.");
}

// returns x * x
static int Square(int x)
{
    return x * x;
}

// returns x * x * x by calling Square
static int Cube(int x)
{
    return Square(x) * x;
}

// returns the area of a circle with the given radius
static double CircleArea(double radius)
```

```
40 {
41     return Math.PI * radius * radius;
42 }
```

Sample run

```
Hello, Elif! Welcome to MCMP113.
Hello, Mehmet! Welcome to MCMP113.
Enter an integer: 7
Square(7) = 49
Cube(7) = 343
Square(3) + Square(4) = 25
Radius in cm: 2.5
Area = 19.63 cm^2
```

	void vs value-returning
purpose	action (print, change an array) vs computation (a result)
`return`	none (`return;` may leave early) vs `return value;` required
call	`Greet("Elif");` vs `int y = Square(5);`

Using a void method as a value is a compile error: `int n = Greet("Ali");` gives CS0029 "Cannot implicitly convert type 'void' to 'int"'. A good rule: methods that compute should *return* the result and not print it; the caller decides what to do with it.

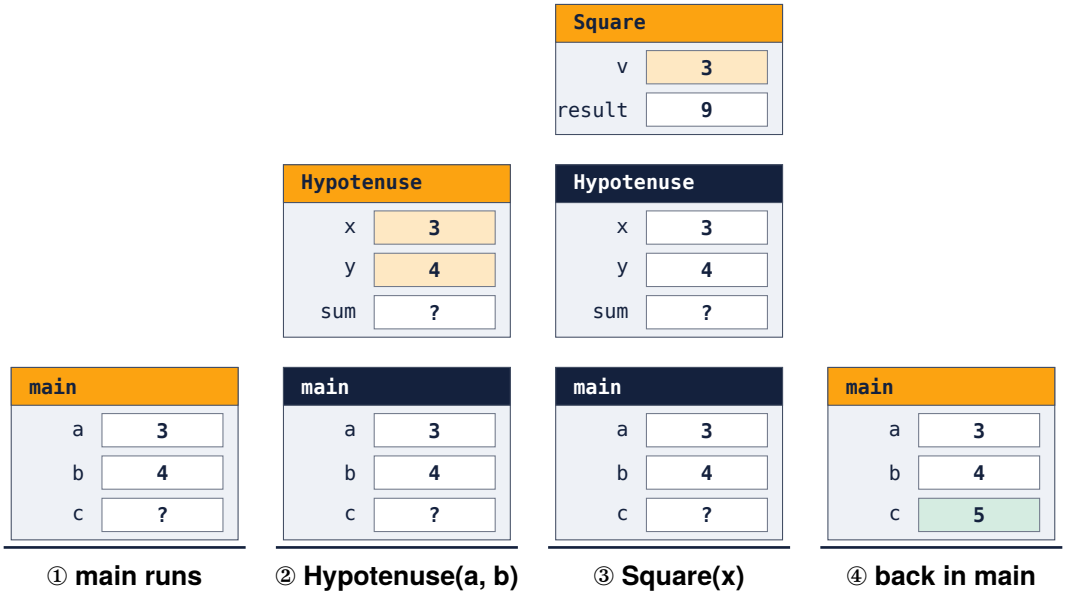
>_ 4 Calling a method: the call stack

4.1 Six steps of a call

What happens in `double c = Hypotenuse(a, b);`?

1. The **arguments are evaluated**: `a` → 3, `b` → 4 (they could be expressions such as `a + 1`).
2. A new **stack frame** is created for `Hypotenuse`: a small memory area with its own variables `x`, `y`, `sum`, `result`. The argument values are **copied** into the parameters: `x` ← 3, `y` ← 4.
3. Execution **jumps** into the method; the caller waits and remembers where to continue (the **return address**).
4. The body runs. It may call other methods (`Square`), which repeat steps 1–6.
5. `return` sends a value back; the call expression is replaced by it: `double c = 5;`.
6. The frame is **removed**; its variables no longer exist.

The frames form a **call stack**: like a stack of plates, the frame put on top last is removed first. The snapshots below show memory while Example 3 runs (newest frame on top, active frame with an amber header, copied arguments in amber).



4.2 Example 3 – a program that prints its own call stack

Listing 3: 12_03_CallStack/Program.cs

```
// MCMP113 · Week 12 · Example 3 — What happens during a call: the call stack
// Each method prints when it starts and when it returns; the indentation shows the call depth.

double a = 3;
double b = 4;
Console.WriteLine($"main: a = {a}, b = {b}");
double c = Hypotenuse(a, b);
Console.WriteLine($"main: c = {c}");

// returns the hypotenuse of a right triangle with legs x and y
static double Hypotenuse(double x, double y)
{
    Console.WriteLine($"  Hypotenuse starts: x = {x}, y = {y}");
    double sum = Square(x) + Square(y);
    double result = Math.Sqrt(sum);
    Console.WriteLine($"  Hypotenuse returns {result}");
    return result;
}

// returns v * v (and reports the call)
static double Square(double v)
{
    Console.WriteLine($"    Square starts: v = {v}");
    double result = v * v;
    Console.WriteLine($"    Square returns {result}");
    return result;
}
```

Output

```
main: a = 3, b = 4
  Hypotenuse starts: x = 3, y = 4
    Square starts: v = 3
    Square returns 9
    Square starts: v = 4
    Square returns 16
  Hypotenuse returns 5
main: c = 5
```

The indentation shows the depth: at most three frames exist at the same time (main, `Hypotenuse`, `Square`). The two `Square` calls do not overlap: the first has returned before the second starts. Both `Hypotenuse` and `Square` have a variable called `result`; they are different boxes in different frames.

4.3 Parameters and arguments

A **parameter** is the variable in the method header (the empty box); an **argument** is the value written in the call (what goes into the box). Arguments are matched to parameters **by position**: the first argument goes into the first parameter. The names do not have to match, but the number and the types must:

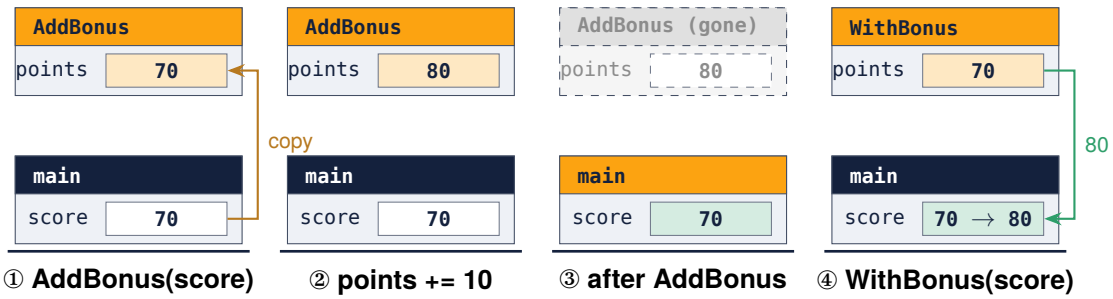
Violation	Real compiler message (.NET 10)
`Square()`	CS7036: There is no argument given that corresponds to the required parameter 'x' of 'Square(int)'
`Square("5")`	CS1503: Argument 1: cannot convert from 'string' to 'int'

An `int` argument may be passed to a `double` parameter (implicit widening, Week 3), but not the other way round.

>_ 5 Passing parameters

5.1 By value: the method gets a copy

For `int`, `double`, `bool`, `char`, `decimal` … C# passes **by value**: the parameter is a new variable that receives a *copy* of the argument. Changing the parameter changes only the copy; the caller's variable is protected. If the caller needs the new value, the method *returns* it.



Listing 4: 12_04_PassByValue/Program.cs

```
// MCMP113 · Week 12 · Example 4 — Passing by value: the method works on a COPY
// Changing a parameter inside a method does not change the caller's variable.

int score = 70;
Console.WriteLine($"Before:            score = {score}");

AddBonus(score);           // 70 is copied into points
Console.WriteLine($"After AddBonus:    score = {score}");
```

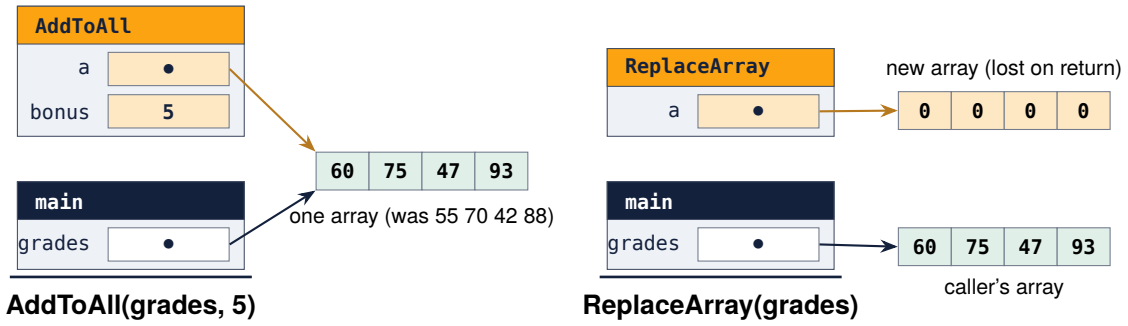
```
9
10 score = WithBonus(score);  // store the returned value
11 Console.WriteLine($"After WithBonus:   score = {score}");
12
13 // adds 10 to its own copy only; the caller's variable does not change
14 static void AddBonus(int points)
15 {
16     points = points + 10;
17     Console.WriteLine($"  inside AddBonus: points = {points}");
18 }
19
20 // returns the value plus 10; the caller decides what to do with it
21 static int WithBonus(int points)
22 {
23     return points + 10;
24 }
```

Output

```
Before:            score = 70
  inside AddBonus: points = 80
After AddBonus:    score = 70
After WithBonus:   score = 80
```

5.2 Arrays as parameters: the reference is copied

An array variable holds a *reference* to the boxes (Week 9: `int[] b = a;` copies no elements). Passing an array copies the reference, not the elements: the parameter and the caller's variable point to the **same** array. So a method can change the caller's *elements* (`a[i] += bonus`), but assigning a *new* array to the parameter (`a = new int[...]`) changes only the parameter; the caller still sees the old array.



Listing 5: 12_05_ArrayParameters/Program.cs

```
1  // MCMP113 · Week 12 · Example 5 — Arrays as parameters: the REFERENCE is copied
2  // The method can change the caller's elements, but giving the parameter a new array does not
       affect the caller.
3
4  int[] grades = { 55, 70, 42, 88 };
5  PrintArray("start", grades);
6
7  AddToAll(grades, 5);                    // changes the elements of grades
8  PrintArray("after AddToAll", grades);
9
10 ReplaceArray(grades);                   // tries to replace the whole array
11 PrintArray("after ReplaceArray", grades);
12
13 // prints a label and the elements of the array
14 static void PrintArray(string label, int[] a)
15 {
```

```
    Console.WriteLine($"{label,-22}: {string.Join(" ", a)}");
}

// adds bonus to every element; the caller sees the change (same array)
static void AddToAll(int[] a, int bonus)
{
    for (int i = 0; i < a.Length; i++)
    {
        a[i] += bonus;
    }
}

// points the parameter to a NEW array; the caller still has the old one
static void ReplaceArray(int[] a)
{
    a = new int[] { 0, 0, 0, 0 };
    PrintArray("  inside ReplaceArray", a);
}
```

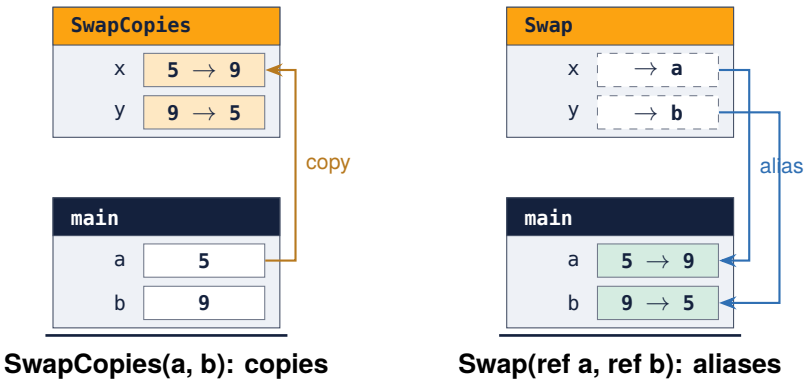
```
Output

start                 : 55 70 42 88
after AddToAll        : 60 75 47 93
  inside ReplaceArray : 0 0 0 0
after ReplaceArray    : 60 75 47 93
```

If a method creates a new array, it should *return* it (Week 13).

5.3 ref and out

With `ref` the parameter has no box of its own: it is an **alias** (another name) of the caller's variable, and every assignment writes into the caller's variable. `ref` must be written in the header *and* in the call (`Swap(ref a, ref b);` forgetting it gives CS1620 "Argument 1 must be passed with the 'ref' keyword"), and the argument must be a variable that already has a value (otherwise CS0165).



SwapCopies(a, b): copies **Swap(ref a, ref b): aliases**

`out` is like `ref`, but for *results*: the caller need not initialise the variable, and the method *must* assign it before it returns (otherwise CS0177 "The out parameter 'q' must be assigned to before control leaves the current method"). You have used an `out` method since Week 5: `int.TryParse(text, out int age)` returns a `bool` (did it work?) and the number through `out`.

Listing 6: 12_06_RefOut/Program.cs

```
// MCMP113 · Week 12 · Example 6 — ref and out parameters
// ref: the parameter is another name for the caller's variable; out: the method must give it a
    value.

int a = 5;
```

```
int b = 9;
Console.WriteLine($"Start:            a = {a}, b = {b}");
SwapCopies(a, b);
Console.WriteLine($"After SwapCopies: a = {a}, b = {b}");
Swap(ref a, ref b);
Console.WriteLine($"After Swap(ref):  a = {a}, b = {b}");

Divide(17, 5, out int q, out int r);
Console.WriteLine($"17 / 5: quotient {q}, remainder {r}");

// int.TryParse is an out method you already know (Week 5)
Console.Write("Your age: ");
if (int.TryParse(Console.ReadLine(), out int age))
{
    Console.WriteLine($"Next year you will be {age + 1}.");
}
else
{
    Console.WriteLine("That is not a whole number.");
}

// swaps its own copies only: the caller's variables do not change
static void SwapCopies(int x, int y)
{
    int temp = x;
    x = y;
    y = temp;
}

// swaps the caller's variables: x and y are aliases (ref)
static void Swap(ref int x, ref int y)
{
    int temp = x;
    x = y;
    y = temp;
}

// gives back two results through out parameters
static void Divide(int dividend, int divisor, out int quotient, out int remainder)
{
    quotient = dividend / divisor;
    remainder = dividend % divisor;
}
```

Sample run 1

```
Start:            a = 5, b = 9
After SwapCopies: a = 5, b = 9
After Swap(ref):  a = 9, b = 5
17 / 5: quotient 3, remainder 2
Your age: 19
Next year you will be 20.
```

Sample run 2

```
Start:            a = 5, b = 9
After SwapCopies: a = 5, b = 9
After Swap(ref):  a = 9, b = 5
17 / 5: quotient 3, remainder 2
Your age: nineteen
That is not a whole number.
```

5.4 Summary: four ways to pass

kind	header	call	what is passed	caller's variable changes?
value	`int x`	`F(n)`	a copy of the value	no
array	`int[] a`	`F(arr)`	a copy of the reference	elements yes, variable no
ref	`ref int x`	`F(ref n)`	the variable itself (alias)	yes
out	`out int x`	`F(out int n)`	the variable; must be assigned	yes (always)

Default choice: value parameters and `return`. Use `ref` only for swap-like methods and `out` when a method must give back several results.

>_ 6 Flexible methods

6.1 Example 7 – optional parameters and named arguments

A parameter with `= value` in the header is **optional**: a call may leave it out, and the default is used. Optional parameters come after the required ones. A **named argument** (`count: 5`) says which parameter receives the value, so earlier optional parameters can be skipped and the order is free. Check the prices: $100 \cdot 1.20 = 120$, $100 \cdot 1.01 = 101$, $100 \cdot 0.75 \cdot 1.20 = 90$, $100 \cdot 0.90 \cdot 1.10 = 99$ (the VAT rates are examples).

Listing 7: 12_07_DefaultNamed/Program.cs

```
// MCMP113 · Week 12 · Example 7 — Optional (default) parameters and named arguments
// Parameters with a default value may be left out; named arguments can skip or reorder them.

PrintLine();                       // both defaults: 20 times '-'
PrintLine('*');                    // count uses its default
PrintLine('=', 12);
PrintLine(count: 5);               // named: skip symbol
PrintLine(count: 8, symbol: '#');  // named: any order

// all net prices are 100 TL; the VAT rates are examples
Console.WriteLine($"Laptop: {Price(100m):F2} TL");                  // default VAT 20 %
Console.WriteLine($"Bread:  {Price(100m, 0.01m):F2} TL");           // VAT 1 %
Console.WriteLine($"Coat:   {Price(100m, discount: 0.25m):F2} TL"); // 25 % off
Console.WriteLine($"Shoes:  {Price(net: 100m, vatRate: 0.10m, discount: 0.10m):F2} TL");

// prints symbol count times; both parameters are optional
static void PrintLine(char symbol = '-', int count = 20)
{
    for (int i = 0; i < count; i++)
    {
        Console.Write(symbol);
    }
    Console.WriteLine();
}

// returns the price after the discount, with VAT added
static decimal Price(decimal net, decimal vatRate = 0.20m, decimal discount = 0m)
{
    decimal discounted = net * (1 - discount);
    return discounted * (1 + vatRate);
}
```

Output

```
--------------------
********************
============
-----
########
Laptop: 120.00 TL
Bread:  101.00 TL
Coat:   90.00 TL
Shoes:  99.00 TL
```

6.2 Example 8 – overloading

Several methods may share a name if their **parameter lists** differ in number or types: this is **overloading** (`Console.WriteLine` has many overloads). The compiler chooses the best match: `Max(3, 2.5)` has no `(int, double)` version, so `3` is widened to `3.0` and `Max(double, double)` is used. The return type alone cannot distinguish overloads. **Local functions cannot be overloaded** (CS0128 "A local variable or function named 'Max' is already defined in this scope"); therefore these methods are written inside `partial class Program`, the class the compiler creates for the top-level statements anyway.

Listing 8: 12_08_Overloading/Program.cs

```
// MCMP113 · Week 12 · Example 8 — Method overloading: one name, different parameter lists
// Local functions cannot be overloaded (error CS0128), so these methods are written in the Program
     class.

int[] scores = { 72, 93, 64, 85 };
Console.WriteLine($"Max(3, 8)        = {Max(3, 8)}");
Console.WriteLine($"Max(2.5, 1.75)   = {Max(2.5, 1.75)}");
Console.WriteLine($"Max(4, 11, 7)    = {Max(4, 11, 7)}");
Console.WriteLine($"Max(scores)      = {Max(scores)}");
Console.WriteLine($"Max(3, 2.5)      = {Max(3, 2.5)}");   // 3 becomes 3.0

// the compiler puts the top-level statements into a class called Program;
// "partial" lets us add our own methods to that class
partial class Program
{
    // returns the larger of two ints
    static int Max(int a, int b)
    {
        return a > b ? a : b;
    }

    // returns the larger of two doubles
    static double Max(double a, double b)
    {
        return a > b ? a : b;
    }

    // returns the largest of three ints (reuses Max)
    static int Max(int a, int b, int c)
    {
        return Max(Max(a, b), c);
    }

    // returns the largest element of an array (champion pattern, Week 10)
    static int Max(int[] values)
    {
        int best = values[0];
        for (int i = 1; i < values.Length; i++)
        {
            if (values[i] > best)
            {
                best = values[i];
            }
        }
        return best;
    }
}
```

Output

```
Max(3, 8)        = 8
Max(2.5, 1.75)   = 2.5
Max(4, 11, 7)    = 11
Max(scores)      = 93
Max(3, 2.5)      = 3
```

>_ 7 Refactoring earlier programs

Refactoring means improving the structure of working code without changing what it does. Algorithms from Weeks 1–11 become reusable methods.

7.1 Example 9 – a math toolkit

`IsLeapYear` and `IsPrime` return `bool`: they answer questions. `IsPrime` returns as soon as a divisor is found. `Lcm` is built on `Gcd`: $\operatorname{lcm}(a,b) = a/\gcd(a,b) \cdot b$ (dividing first keeps the numbers small): $84/12 \cdot 36 = 252$. `Gcd` changes its parameters `a` and `b`, but the caller's values are unaffected (value parameters).

Listing 9: 12_09_MathToolkit/Program.cs (methods only; the main part prints the table below)

```
// true if year is a leap year in the Gregorian calendar
static bool IsLeapYear(int year)
{
    return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
}

// true if n is prime (trial division up to the square root of n)
static bool IsPrime(int n)
{
    if (n < 2)
    {
        return false;
    }
    for (int d = 2; d * d <= n; d++)
    {
        if (n % d == 0)
        {
            return false;
        }
    }
    return true;
}

// returns n! = 1 * 2 * ... * n as a long (correct up to 20!)
static long Factorial(int n)
{
    long product = 1;
    for (int i = 2; i <= n; i++)
    {
        product *= i;
    }
    return product;
}

// greatest common divisor with Euclid's algorithm
static int Gcd(int a, int b)
{
    while (b != 0)
```

```
    {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

// least common multiple, built on Gcd
static int Lcm(int a, int b)
{
    return a / Gcd(a, b) * b;
}
```

Output

```
1900: not a leap year
2000: leap year
2024: leap year
2026: not a leap year
Primes up to 30: 2 3 5 7 11 13 17 19 23 29
5! = 120, 10! = 3628800, 20! = 2432902008176640000
Gcd(84, 36) = 12, Lcm(84, 36) = 252
Gcd(17, 5) = 1, Lcm(4, 6) = 12
```

7.2 Example 10 – input methods

`ReadInt` contains the `TryParse` loop of Week 7 *once*; `ReadIntInRange` calls it and adds the range check. `Average` uses a cast to avoid integer division: $(70 + 85 + 40)/3 = 65.00$.

Listing 10: 12_10_ReadValidated/Program.cs

```
// MCMP113 · Week 12 · Example 10 — Input methods: ReadInt and ReadIntInRange
// The TryParse loop is written once and reused for every input; Average works for any int array.

int n = ReadIntInRange("How many students (1-10)? ", 1, 10);
int[] grades = new int[n];
for (int i = 0; i < n; i++)
{
    grades[i] = ReadIntInRange($"Grade {i + 1} (0-100): ", 0, 100);
}
Console.WriteLine($"Grades: {string.Join(", ", grades)}");
Console.WriteLine($"Average: {Average(grades):F2}");

// asks until the user types a whole number, then returns it
static int ReadInt(string prompt)
{
    Console.Write(prompt);
    int value;
    while (!int.TryParse(Console.ReadLine(), out value))
    {
        Console.Write("  Whole number, please: ");
    }
    return value;
}

// asks until the number is between min and max (both included)
static int ReadIntInRange(string prompt, int min, int max)
{
    int value = ReadInt(prompt);
    while (value < min || value > max)
    {
        Console.WriteLine($"  Must be between {min} and {max}.");
```

```
        value = ReadInt(prompt);
    }
    return value;
}

// returns the average of the elements of an int array
static double Average(int[] values)
{
    int sum = 0;
    for (int i = 0; i < values.Length; i++)
    {
        sum += values[i];
    }
    return (double)sum / values.Length;
}
```

Sample run

```
How many students (1-10)? abc
  Whole number, please: 12
  Must be between 1 and 10.
How many students (1-10)? 3
Grade 1 (0-100): 105
  Must be between 0 and 100.
Grade 1 (0-100): 70
Grade 2 (0-100): 85
Grade 3 (0-100): -5
  Must be between 0 and 100.
Grade 3 (0-100): 40
Grades: 70, 85, 40
Average: 65.00
```

7.3 Example 11 – an array toolkit

The statistics loops of Weeks 9–10 become `PrintArray`, `Average`, `Max`, `Min` and `CountAtLeast`; `Report` calls all of them and is itself called twice, for two different arrays. Group A: $399/6 = 66.50$, range $95.5 - 38 = 57.5$. The methods `Min` and `CountAtLeast` (end of the file) follow the same patterns as `Max` and the counting loop of Week 10.

Listing 11: 12_11_ArrayToolkit/Program.cs (excerpt)

```
double[] groupA = { 72.5, 45, 88, 60, 95.5, 38 };
double[] groupB = { 80, 65, 90 };

Report("Group A", groupA);
Report("Group B", groupB);

// prints a short statistics report for one group (calls the other methods)
static void Report(string title, double[] grades)
{
    Console.WriteLine($"--- {title} ---");
    PrintArray(grades);
    Console.WriteLine($"Average: {Average(grades):F2}");
    double max = Max(grades);
    double min = Min(grades);
    Console.WriteLine($"Max: {max}, Min: {min}, range: {max - min}");
    int passed = CountAtLeast(grades, 50);
    Console.WriteLine($"Passed (>= 50): {passed} of {grades.Length}");
}

// prints the elements on one line, separated by spaces
static void PrintArray(double[] a)
```

```
{
    Console.WriteLine("Grades: " + string.Join(" ", a));
}

// returns the arithmetic mean of the elements
static double Average(double[] a)
{
    double sum = 0;
    foreach (double x in a)
    {
        sum += x;
    }
    return sum / a.Length;
}

// returns the largest element (champion pattern)
static double Max(double[] a)
{
    double best = a[0];
    for (int i = 1; i < a.Length; i++)
    {
        if (a[i] > best)
        {
            best = a[i];
        }
    }
    return best;
}
```

Output

```
--- Group A ---
Grades: 72.5 45 88 60 95.5 38
Average: 66.50
Max: 95.5, Min: 38, range: 57.5
Passed (>= 50): 4 of 6
--- Group B ---
Grades: 80 65 90
Average: 78.33
Max: 90, Min: 65, range: 25
Passed (>= 50): 3 of 3
```

7.4 Example 12 – a menu calculator

The `do-while` + menu program of Week 7, organised with methods. The main loop only decides *what* to do; `ShowMenu`, `ReadIntInRange`, `ReadDouble` and `Calculate` do the work. `Calculate` returns the result of a switch expression (Week 5).

Listing 12: 12_12_MenuCalculator/Program.cs (main part)

```
int choice;
do
{
    ShowMenu();
    choice = ReadIntInRange("Choice: ", 0, 5);
    if (choice != 0)
    {
        double x = ReadDouble("First number:  ");
        double y = ReadDouble("Second number: ");
        if (choice == 4 && y == 0)
        {
            Console.WriteLine("Error: division by zero.");
```

```
        }
        else
        {
            double result = Calculate(choice, x, y);
            Console.WriteLine($"Result: {result:F2}");
        }
    }
} while (choice != 0);
Console.WriteLine("Goodbye!");
```

Listing 13: 12_12_MenuCalculator/Program.cs (Calculate; ReadDouble and ReadIntInRange as in Example 10)

```
// returns the result of the chosen operation (switch expression, Week 5)
static double Calculate(int op, double x, double y)
{
    return op switch
    {
        1 => x + y,
        2 => x - y,
        3 => x * y,
        4 => x / y,
        _ => Math.Pow(x, y),
    };
}
```

Sample run

```
1) +   2) -   3) *   4) /   5) power   0) exit
Choice: 1
First number:  12.5
Second number: 7.5
Result: 20.00
1) +   2) -   3) *   4) /   5) power   0) exit
Choice: 4
First number:  10
Second number: 0
Error: division by zero.
1) +   2) -   3) *   4) /   5) power   0) exit
Choice: 7
  Please type 0-5: 5
First number:  2
Second number: 10
Result: 1024.00
1) +   2) -   3) *   4) /   5) power   0) exit
Choice: 0
Goodbye!
```

>_ 8 Debugging calls in VS Code

Example 13 is a small program made for the debugger.

1. Click left of the line number of `int sum = DigitSum(number);` to set a **breakpoint** (red dot; key F9) and start with F5. The program stops *before* that line.
2. **F10 (Step Over)** runs the whole call at once. **F11 (Step Into)** enters `DigitSum`; the VARIABLES panel shows `n` and `sum` at every step, and the CALL STACK panel shows `DigitSum` on top of `<Main>$` – our stack diagrams. **Shift+F11 (Step Out)** finishes the method and returns to the caller.

Listing 14: 12_13_DigitTools/Program.cs (DigitSum; CountDigits and Reverse are similar)

```
25 // returns the sum of the digits of n
26 static int DigitSum(int n)
27 {
28     int sum = 0;
29     while (n > 0)
30     {
31         sum += n % 10;
32         n /= 10;
33     }
34     return sum;
35 }
```

DigitSum(2026)	n % 10	sum	new n
start	–	0	2026
1st pass	6	6	202
2nd pass	2	8	20
3rd pass	0	8	2
4th pass	2	10	0

Sample run

```
Enter a positive integer: 2026
Digits: 4, digit sum: 10, reversed: 6202
Average digit: 2.50
```

>_ 9 Common mistakes

1. Wrong number or type of arguments

`Square()` (CS7036), `Square("5")` (CS1503). Match the parameter list by position.

2. Using a void method as a value

`int n = Greet("Ali");` gives CS0029. A void method is called as a statement.

3. Ignoring the returned value

`Square(n); Console.WriteLine(n);` compiles but prints the old `n`: the result was thrown away. Write `n = Square(n);`.

4. Expecting a value parameter to change the caller

`AddBonus(score)` changes only its copy. Return the new value, or (rarely) use `ref`.

5. ref only in the header

`Swap(a, b);` for `Swap(ref int x, ref int y)` gives CS1620; write `ref` in the call too.

6. Two local functions with the same name

CS0128: local functions cannot be overloaded. Use different names, or methods in `partial class Program`.

7. Printing inside a “compute” method

A method that prints its result cannot be reused in a calculation. Return the value; print in the caller.

>_ 10 Summary

- A method is a named block of code: write once, call many times. Header: `static`, return type (or `void`), PascalCase name, parameter list. Local functions go at the bottom of `Program.cs`.
- Every call creates a stack frame; arguments are evaluated and *copied* into the parameters; `return` sends a value back and the frame is removed.
- Value parameters never change the caller's variable. Arrays pass a copy of the reference: elements can change, the variable cannot. `ref` makes an alias, `out` returns extra results.
- Optional parameters, named arguments and overloading make methods flexible; local functions cannot be overloaded.
- Refactor repeated code into methods; debug calls with F9, F10, F11 and Shift+F11.

>_ 11 Glossary (English – Turkish)

English	Turkish	Meaning
method / function	metot / fonksiyon	named block of code that does one job
local function	yerel fonksiyon	method declared inside other code (top-level statements)
call (invoke)	çağırmak	run a method: `Square(5)`
parameter	parametre	variable in the method header
argument	argüman (gerçek parametre)	value written in the call
return value	dönüş değeri	value sent back by `return`
return type	dönüş tipi	type of the result, or `void`
call stack	çağrı yığını	list of active method frames
stack frame	yığın çerçevesi	memory of one call (parameters, locals)
pass by value	değer ile aktarım	the method receives a copy
pass by reference (`ref`)	referans ile aktarım	the parameter is an alias of the variable
output parameter (`out`)	çıkış parametresi	extra result written by the method
optional / default parameter	isteğe bağlı / varsayılan parametre	may be left out in the call
named argument	adlandırılmış argüman	`count: 5`
overloading	aşırı yükleme	same name, different parameter lists
refactoring	yeniden düzenleme	better structure, same behaviour
breakpoint	kesme noktası	line where the debugger stops
step into / over / out	içine gir / üzerinden geç / dışına çık	debugger commands F11 / F10 / Shift+F11

>_ 12 Exercises

12.1 Check yourself: predict the output

Write the exact output, drawing the stack frames. Then run the code to check.

Q1

```
int x = 5;
Change(x);
Console.WriteLine(x);
x = Twice(x);
Console.WriteLine(x);

static void Change(int x)
{
    x = x * 100;
}

static int Twice(int n)
{
    return n * 2;
}
```

Q2

```
int[] a = { 1, 2, 3 };
int k = 10;
Update(a, k);
Console.WriteLine($"{a[0]} {k}");
Bump(ref k);
Console.WriteLine(k);

static void Update(int[] arr, int n)
{
    arr[0] = 99;
    n = 0;
}

static void Bump(ref int n)
{
    n++;
}
```

Q3

```
Console.WriteLine(
    F(2) + F(3));

static int F(int n)
{
    Console.Write($"F({n}) ");
    return n * n + 1;
}
```

12.2 In-class exercises (pairs)

E1. ⋆ **Rectangle.** Write `Area(w, h)` and `Perimeter(w, h)` returning `double`. Read `w` and `h` and print both with `F2`.

E2. ⋆ **Box of symbols.** Write `PrintBox(width, height, symbol = '#')` that prints a filled rectangle by calling `PrintLine` from Example 1. Test `PrintBox(5, 3)` and `PrintBox(8, 2, '*')`.

E3. ⋆⋆ **Temperature table.** Write `CelsiusToFahrenheit(c)` ($F = C \cdot 9/5 + 32$) and print a table for $-10, 0, 10, \ldots, 40\,^{\circ}\mathrm{C}$.

E4. ⋆⋆ **Sort three.** Write `SortThree(ref int a, ref int b, ref int c)` that puts three variables in ascending order using `Swap(ref, ref)`.

12.3 Homework

H1. ⋆⋆ **Days in a month.** Write `DaysInMonth(month, year)` using `IsLeapYear` from Example 9. Print the days of all 12 months of 2024 and their total.

H2. ⋆ **Power.** Write `Power(double x, int exponent = 2)` with a loop (no `Math.Pow`), for exponents ≥ 0. Test `Power(3)`, `Power(2, 10)`, `Power(1.5, 3)`, `Power(7, 0)`.

H3. ⋆⋆ **Area overloads.** Write three overloads `Area(r)` (circle), `Area(w, h)` (rectangle) and

`Area(a, b, c)` (triangle, Heron: $s = \frac{a+b+c}{2}$, $A = \sqrt{s(s-a)(s-b)(s-c)}$).

H4. ⋆⋆ **Sorting method.** Turn the bubble sort of Week 10 into `void BubbleSort(int[] a)` and add `PrintArray(int[] a)`. Sort `{ 31, 12, 47, 8, 25, 19 }`.

12.4 Extra practice

P1. ⋆ Which are parameters and which are arguments in `double c = Hypotenuse(a + 1, 2 * b);` and `static double Hypotenuse(double x, double y)`? What values do the parameters get if `a = 3, b = 4`?

P2. ⋆ Find the bug: `int n = 6; Square(n); Console.WriteLine(n);` is meant to print 36.

P3. ⋆⋆ While `Lcm(84, 36)` of Example 9 is computing, how many frames are on the call stack at most? Name them from bottom to top.

P4. ⋆⋆ Why does `Swap(int[] arr, int i, int j)` swap two *elements* without `ref`? Write it and test it on `{ 10, 20, 30 }` with `i = 0, j = 2`.

P5. ⋆⋆⋆ Write `MinMax(int[] values, out int min, out int max)` and test it on `{ 7, -3, 12, 0, 5 }`.

>_ 13 References

- Microsoft Learn — Methods (C# programming guide): https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/methods
- Microsoft Learn — Local functions: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/local-functions
- Microsoft Learn — Method parameters (`ref`, `out`, `in`): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/method-parameters
- Microsoft Learn — Named and optional arguments: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/named-and-optional-arguments
- Microsoft Learn — Top-level statements: https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/program-structure/top-level-statements
- VS Code — Debugging: https://code.visualstudio.com/docs/editor/debugging